

Fullstack Development

Preflight project - backend

[Github Repo](#)

Backend applications

- Data storage
- Business logic
- Authentication / authorization
- APIs for frontend frameworks (*depending on architecture*)
- Connection to other services

Node JS backend frameworks

- [Github stars](#)
- [State of JS 2025](#)

Packages

- `pnpm init`
- Express JS
 - `pnpm i express cors helmet morgan debug`
- Typescript
 - `pnpm i typescript @tsconfig/node-lts @tsconfig/node-ts tsx tsc-alias`
 - `pnpm i -D @types/cors @types/express @types/debug @types/morgan`
`@types/node cross-env nodemon`
- ORM
 - `pnpm i drizzle-orm postgres dotenv`
 - `pnpm i -D drizzle-kit`

Package explanation

Package	Details
<code>express</code>	Express (<i>duh!</i>)
<code>helmet</code>	Set default HTTP response header
<code>cors</code>	Enable Cross-origin resource sharing (CORS)
<code>morgan</code>	HTTP request logger
<code>debug</code>	Debugging utility
<code>nodemon</code>	Script monitoring during dev

ORM code

- All [files](#) in `./db` folder.
 - You can change schema.
- `./drizzle.config.ts` [\(Link\)](#)

Files

-  `./.env` (Copy from [here](#))
-  `./nodemon.json` ([Link](#))
-  `./.gitignore` ([Link](#))
-  `./tsconfig.json` ([Link](#))
- Scripts in `package.json` ([Link](#))

Minimal example

-  `./src/index.ts` [\(Link\)](#)
- Sync database (if you have not done so)
 - `pnpm run db:push`
- Start dev
 - `pnpm run dev`

Full example

-  `./src/index.ts` [\(Link\)](#)
- Build
 - `pnpm run build`
- Start production
 - `pnpm run start`

API Spec

- Insomnia
- Postman

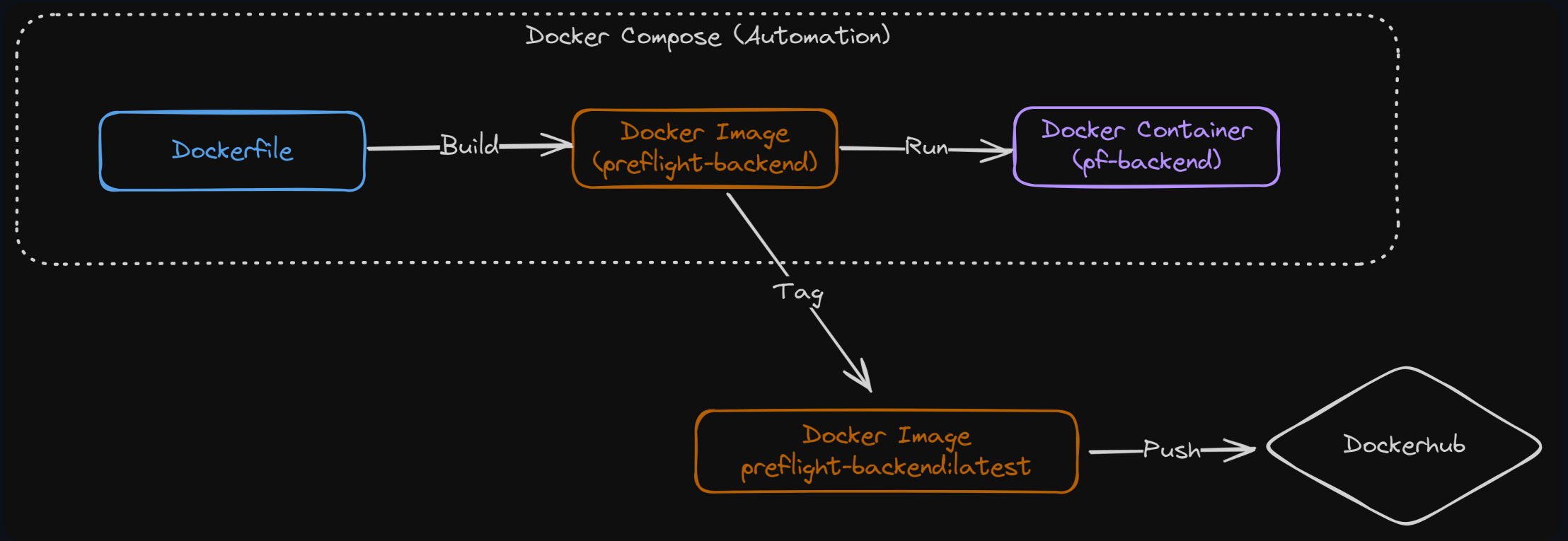
Debugging (optional)

-  `./.vscode/launch.json` ([Link](#))

Migration

- Run before the containerization step.
- `pnpm run db:generate`

Containerization



Steps

-  `./Dockerfile` [\(Link\)](#)
-  `./dockerignore` [\(Link\)](#)
-  `./docker-compose.yml` [\(Link\)](#)
-  `./.env.test` from `./.env.test.example` [\(Link\)](#)
-  `docker compose --env-file ./env.test up -d --force-recreate --build`

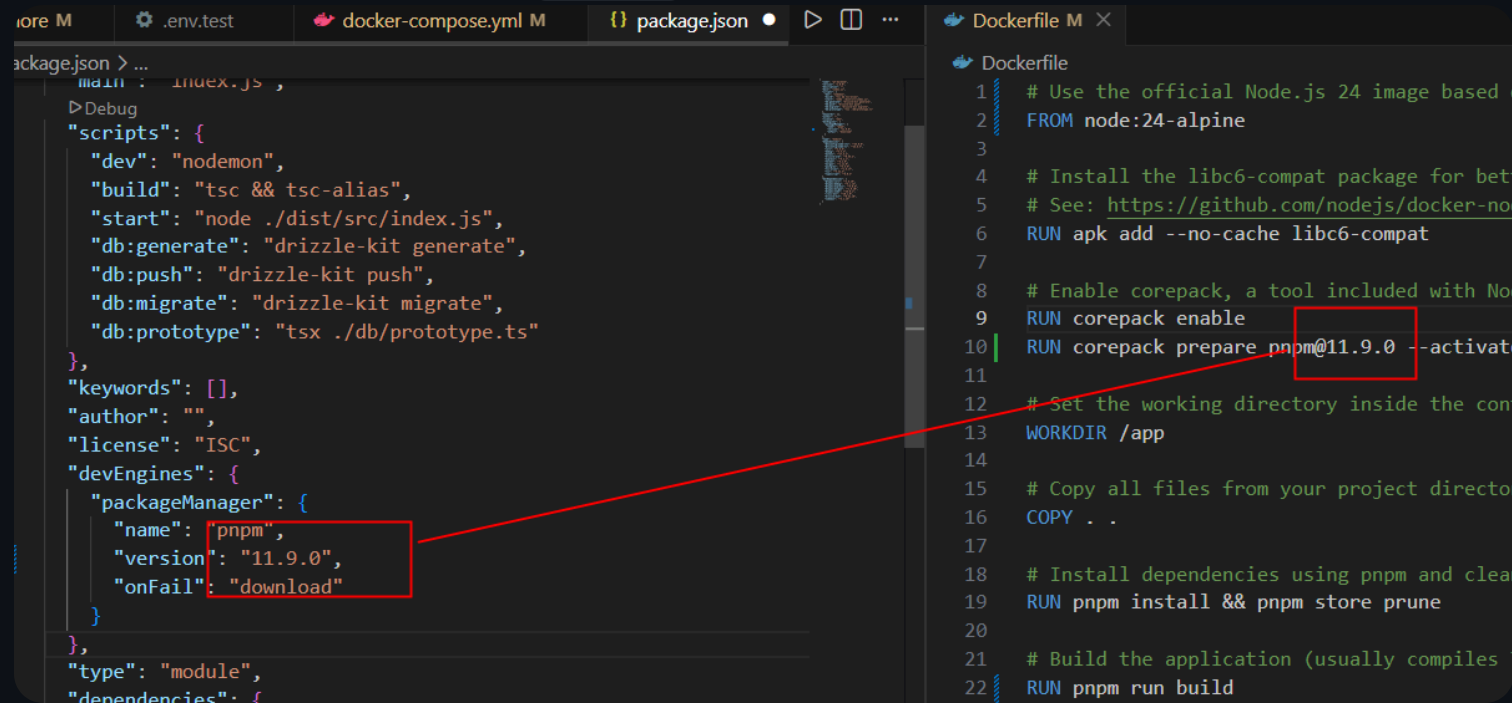
Corepack

- In a `Node-Alpine` Dockerfile, `Corepack` is an official Node.js tool used to install and manage package managers like `pnpm` and `yarn`.
- Corepack require an exact SemVer format (e.g., `10.0.0` or `pnpm@9.1.0`).
 - They will crash if they encounter a range modifier or invalid syntax.

```
-  "packageManager": {  
+    "name": "pnpm",  
-    "version": "^11.9.0",  
+    "version": "11.9.0",  
    "onFail": "download"
```

Dockerfile

- Double check the **pnpm** version in your local machine and the Dockerfile.



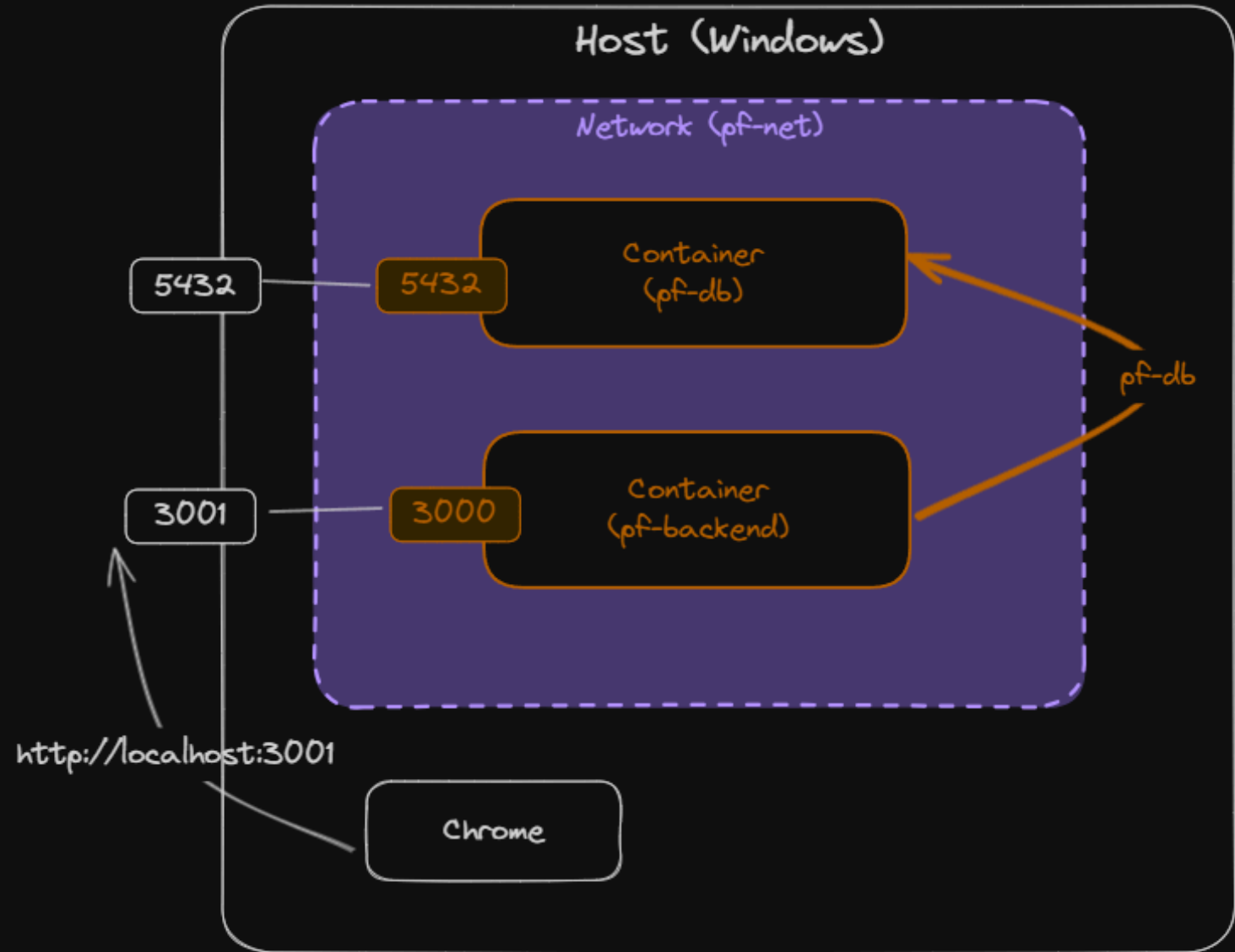
The screenshot shows a code editor with two files open: `package.json` and `Dockerfile`. A red line connects the `pnpm@11.9.0` version specified in the `packageManager` field of `package.json` to the `pnpm@11.9.0` version used in the `Dockerfile` command.

```
package.json > ...
main: index.js,
scripts: {
  dev: nodemon,
  build: tsc && tsc-alias,
  start: node ./dist/src/index.js,
  db:generate: drizzle-kit generate,
  db:push: drizzle-kit push,
  db:migrate: drizzle-kit migrate,
  db:prototype: tsx ./db/prototype.ts
},
keywords: [],
author: "",
license: ISC,
devEngines: {
  packageManager: {
    name: pnpm,
    version: 11.9.0,
    onFail: download
  }
},
type: module,
dependencies: {
```

```
Dockerfile
1 # Use the official Node.js 24 image based on
2 FROM node:24-alpine
3
4 # Install the libc6-compat package for better compatibility
5 # See: https://github.com/nodejs/docker-node/blob/main/docs/compatibility.md
6 RUN apk add --no-cache libc6-compat
7
8 # Enable corepack, a tool included with Node.js that lets you use
9 # any package manager
10 RUN corepack enable
11 RUN corepack prepare pnpm@11.9.0 --activate
12
13 # Set the working directory inside the container
14 WORKDIR /app
15
16 # Copy all files from your project directory to the container
17 COPY . .
18
19 # Install dependencies using pnpm and clean up
20 RUN pnpm install && pnpm store prune
21
22 # Build the application (usually compiles TypeScript)
23 RUN pnpm run build
```

Network

- Notice the port `3001 / 3000`.
- *Can you find where these ports are specified?*



Automated database migration

- We want to make sure that database migrations are run automatically every time the container starts.
- We use the `post_start` hook in `docker-compose.yml`

Dockerhub

- Create an account at <https://hub.docker.com>.
- Create repository called `preflight-backend`.
- Login to your account in terminal
 - `docker login`

Push to Dockerhub

- Tag image

-  `docker tag preflight-backend [DOCKERHUB_ACCOUNT]/preflight-backend:latest`

- `docker login`

- Push image

-  `docker push [DOCKERHUB_ACCOUNT]/preflight-backend:latest`

Useful docker commands

- Inspect

- `docker ps`
- `docker network ls`
- `docker volume ls`

- Cleaning

- `docker image prune -a`
- `docker builder prune`
- `docker volume prune`
- `docker network prune`